PES-VCS Lab Report

Building a Version Control System from Scratch

Student	Atharva Ashish Vyas
SRN	PES1UG24CS093
Course	Operating Systems Lab
Platform	Ubuntu 22.04
GitHub Repository	https://github.com/Atharva-Ashish-Vyas/PES1UG24CS093-pes-vcs

Phase 1: Object Storage Foundation

Implemented the content-addressable object store in **object.c**. Every piece of data (blobs, trees, commits) is stored under .pes/objects/XX/YYY... where XX is the first two hex characters of the SHA-256 hash. The two key functions are:

object_write — prepends a type header (*blob <size>\0*), computes SHA-256 of the full object, deduplicates, and writes atomically using a temp-file-then-rename pattern with fsync for durability.

object_read — reads the file, re-computes the hash for integrity verification, parses the header, and returns the data portion.

Screenshot 1A — test_objects output

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Screenshot 1B — Sharded object store

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Phase 2: Tree Objects

Implemented **tree_from_index** in **tree.c**. A tree object represents a directory snapshot in binary format. Each entry is <mode> <name>\0<32-byte-hash>. The recursive helper **write_tree_level** groups index entries by directory prefix using strchr('/'), recursing into subdirectories to build nested subtrees before writing the root tree.

Screenshot 2A — test_tree output

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Screenshot 2B — Raw tree object hex dump

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ find .pes/objects -type f | sort
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ ./test_objects
xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Phase 3: The Index (Staging Area)

Implemented **index_load**, **index_save**, and **index_add** in **index.c**. The index is a human-readable text file at .pes/index with one line per staged file in the format: <mode> <hex-hash> <mtime> <size> <path>.

index_load — reads the text file with fscanf, initialising an empty index if the file doesn't exist yet (not an error).

index_save — sorts entries by path, writes to a temp file, fflush+fsync, then atomically renames. The Index struct is heap-allocated to avoid a ~5 MB stack overflow (10,000 entries × 528 bytes each).

index_add — reads the file, writes it as OBJ_BLOB, records mtime and size for fast change detection, then updates or appends the index entry.

Screenshot 3A — pes init → pes add → pes status

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
[9868aeb] Initial commit
Committed: 9868aeb65c3c... Initial commit
[74c1bcb] Add world
Committed: 74c1bcbc7a66... Add world
[d79bc9b] Add farewell
Committed: d79bc9b5e9f1... Add farewell
commit d79bc9b5e9f1d3c10bb559afba5461a41a4827302f21ac1e3be5aeb38cee5fbc
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664449

    Add farewell


commit 74c1bcbc7a6628bdfc2b9e2caa7afc4656a00574a9d6e7b743c04e97e56da963
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664449

    Add world
!

commit 9868aeb65c3c833d6767a74be25b0859d310925c12fe0fb950a4360653e71783
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664449

    Initial commit
ialA
```

Screenshot 3B — cat .pes/index

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776664279 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776664279 6 file2.txt
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Phase 4: Commits and History

Implemented **commit_create** in **commit.c**. A commit object is a text object containing tree hash, optional parent hash, author, timestamp, and message — separated from the message by a blank line.

The sequence: **(1)** call *tree_from_index()* to snapshot staged files, **(2)** call *head_read()* to get the parent (none for first commit), **(3)** fill the Commit struct with author from $PES_AUTHOR and time(NULL), **(4)** serialize and write as OBJ_COMMIT, **(5)** update HEAD atomically with *head_update()*.

Screenshot 4A — pes log (three commits)

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/74/c1bcbc7a6628bdfc2b9e2caa7afc4656a00574a9d6e7b743c04e97e56da963
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/98/68aeb65c3c833d6767a74be25b0859d310925c12fe0fb950a4360653e71783
.pes/objects/d7/9bc9b5e9f1d3c10bb559afba5461a41a4827302f21ac1e3be5aeb38cee5fbc
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Screenshot 4B — Object store growth

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
d79bc9b5e9f1d3c10bb559afba5461a41a4827302f21ac1e3be5aeb38cee5fbc
ref: refs/heads/main
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Screenshot 4C — Reference chain

```
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
d79bc9b5e9f1d3c10bb559afba5461a41a4827302f21ac1e3be5aeb38cee5fbc
ref: refs/heads/main
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Final: Integration Test

The full integration test (make test-integration) runs **test_sequence.sh** which exercises the complete workflow: init → add → commit × 3 → log → verify reference chain → count objects.

```
--- Full History ---
commit c0cb9baf4be091cc81a7a272f93631e85af6dafea8bf3932714723332afff69b
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664753

    Add farewell


commit 9ce5596763f248ce3b2cc863b80e52ffffe3b96b19ffb38de9eb3f056d2105e5
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664753

    Update file.txt


commit 898433a30a4a9ede69b265b316ddf5951d1d33caba886698116c2c596ef5bcc3
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664753

    Initial commit



--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
c0cb9baf4be091cc81a7a272f93631e85af6dafea8bf3932714723332afff69b

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/89/8433a30a4a9ede69b265b316ddf5951d1d33caba886698116c2c596ef5bcc3
.pes/objects/9c/e5596763f248ce3b2cc863b80e52ffffe3b96b19ffb38de9eb3f056d2105e5
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c0/cb9baf4be091cc81a7a272f93631e85af6dafea8bf3932714723332afff69b
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```


```
--- Full History ---
commit c0cb9baf4be091cc81a7a272f93631e85af6dafea8bf3932714723332afff69b
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664753

    Add farewell


commit 9ce5596763f248ce3b2cc863b80e52ffffe3b96b19ffb38de9eb3f056d2105e5
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664753

    Update file.txt


commit 898433a30a4a9ede69b265b316ddf5951d1d33caba886698116c2c596ef5bcc3
Author: Atharva Ashish Vyas <PES1UG24CS093>
Date:   1776664753

    Initial commit



--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
c0cb9baf4be091cc81a7a272f93631e85af6dafea8bf3932714723332afff69b

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/89/8433a30a4a9ede69b265b316ddf5951d1d33caba886698116c2c596ef5bcc3
.pes/objects/9c/e5596763f248ce3b2cc863b80e52ffffe3b96b19ffb38de9eb3f056d2105e5
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c0/cb9baf4be091cc81a7a272f93631e85af6dafea8bf3932714723332afff69b
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
atharva@atharva-VirtualBox:~/PES1UG24CS093-pes-vcs$
```

Phase 5: Branching and Checkout — Analysis

Q5.1 — Implementing pes checkout

Files that change in .pes/:

Only .pes/HEAD needs to change. It is updated from ref: refs/heads/old-branch to ref: refs/heads/new-branch. If the branch doesn't exist, the file .pes/refs/heads/<branch> must be created containing the current commit hash.

What must happen to the working directory:

The working directory files must be updated to match the target branch's tree. This means: (1) read the target branch's commit hash from .pes/refs/heads/<branch>, (2) read that commit object to get its tree hash, (3) recursively walk the tree, (4) for each file in the tree, write the blob contents to disk, and (5) update the index to reflect the new branch's staged state.

What makes this complex:

The operation is complex because it must handle three classes of files simultaneously: files that exist in the target branch but not the current one (must be created), files in the current branch but not the target (must be deleted), and files that exist in both but differ (must be overwritten). Each of these must be reconciled with the user's uncommitted changes without data loss. Additionally, the operation must be atomic — a crash partway through would leave the repository in an inconsistent state. Git solves this with careful ordering: update tracked files first, then update HEAD last so a crash before HEAD update leaves the old branch intact.

Q5.2 — Detecting dirty working directory conflicts

To detect whether a file has uncommitted changes that would conflict with a checkout, we use only the index and the object store:

Step 1 — Check if working file differs from index: For each file tracked in the index, call stat() on the working directory copy. Compare st_mtime and st_size against the index entry's stored mtime and size. If either differs, the file is potentially modified. For certainty, read the file, hash it with SHA-256, and compare the hash against the index entry's blob hash.

Step 2 — Check if the file differs between branches: Read the target branch's commit → tree → find the same file path → get its blob hash. Compare this blob hash against the current index entry's blob hash. If they differ, the branches have different versions of this file.

Conflict condition: If *both* conditions are true — the working file differs from the index AND the two branches have different versions — then checkout must refuse with an error like: *"error: Your local changes to 'file.txt' would be overwritten by checkout."* If only the branch versions differ but the working file matches the index, checkout can safely overwrite it.

Q5.3 — Detached HEAD state

What is detached HEAD: Normally HEAD contains a symbolic reference like ref: refs/heads/main. In detached HEAD state, HEAD contains a raw commit hash directly, e.g. a1b2c3d4e5f6.... This happens when you checkout a specific commit rather than a branch name.

What happens when you commit in detached HEAD: New commits are created and stored in the object store normally. HEAD is updated to point directly to each new commit hash. However, since no branch reference points to these commits, they are not reachable from any branch. If you switch to another branch, these commits become unreachable — they exist in the object store but nothing points to them, making them candidates for garbage collection.

How to recover: If you know the commit hash (visible in the terminal before switching away), you can create a new branch pointing to it: pes branch recovery-branch <hash>, which creates .pes/refs/heads/recovery-branch containing that hash. The commits are then reachable again. If you have lost the hash, you would need to search the object store manually or use a reflog (which PES-VCS does not implement, but Git's .git/logs/HEAD records every HEAD movement for exactly this recovery scenario).

Phase 6: Garbage Collection — Analysis

Q6.1 — Algorithm to find and delete unreachable objects

Algorithm (mark-and-sweep):

Phase 1 — Mark reachable objects: Start from all branch tips by reading every file in .pes/refs/heads/. For each branch, walk the commit chain following parent pointers until reaching the root commit. For each commit visited, add its hash to a reachable set, then recursively traverse its tree object — adding every tree hash and every blob hash encountered. Use a hash set (e.g. a hash table or a sorted array of 32-byte hashes) to track visited objects and avoid re-visiting shared objects.

Phase 2 — Sweep unreachable objects: Walk every file under .pes/objects/ using opendir/readdir recursively. For each object file, reconstruct its hash from its path (shard dir + filename). If this hash is NOT in the reachable set, delete the file with unlink(). Optionally remove empty shard directories with rmdir().

Data structure: A hash set implemented as a sorted array of 32-byte hashes supports O(log n) membership queries via binary search. Alternatively, a hash table with open addressing gives O(1) average lookup. For 100,000 commits and 50 branches, the set might hold up to a few million hashes, so memory efficiency matters.

Estimation for 100,000 commits and 50 branches: Each commit object references 1 tree. Each tree might reference 10–50 blobs and sub-trees on average. Assuming ~20 objects per commit on average: 100,000 commits × 20 = approximately **2,000,000 objects** to visit during the mark phase. Many blobs will be shared across commits (unchanged files), so the actual unique reachable set will be smaller — perhaps 500,000–800,000 unique objects. The sweep phase visits all files on disk, which includes both reachable and unreachable objects.

Q6.2 — Race condition between GC and concurrent commit

The race condition: Consider this interleaving between a commit operation (Thread C) and garbage collection (Thread GC):

- 1. Thread C calls object_write() for a new blob — the blob is written to disk but the index has NOT beenupdated yet.
- 2. Thread GC starts its mark phase — it reads all branch refs and walks all reachable objects. The newblob is not reachable from any branch yet (index not saved), so GC does NOT add it to the reachable set.
- 3. Thread GC enters the sweep phase and deletes the new blob (it appears unreachable).
- 4. Thread C finishes: saves the index (referencing the deleted blob hash), builds the tree, writes thecommit. The commit now references a blob that no longer exists in the object store — the repository is corrupt.

How Git's real GC avoids this: Git uses several strategies:

(1) Grace period: Git's git gc only deletes objects that are older than a configurable grace period (default: 2 weeks, controlled by gc.pruneExpire). Newly written objects are never deleted even if temporarily unreachable, giving ongoing operations time to complete.

(2) Loose object safety: Git writes new objects as loose files before updating any references. GC checks the mtime of object files — recently created objects (within the grace period) are preserved regardless of reachability.

(3) **Pack file locking:** When GC repacks objects into pack files, it uses lock files (.keep files) to prevent concurrent writers from being affected mid-operation.

(4) **Ref locking:** Reference updates use lock files (refs/heads/main.lock) so GC can safely read a consistent snapshot of all branch tips without a branch being updated underneath it.